

Chương 3 đã trình bày các công nghệ nền tảng. Chương này trình bày cách thiết kế và hiện thực hệ thống dựa trên các công nghệ đó, gồm: kiến trúc tổng thể, thiết kế các mô-đun chính, máy trạng thái điều khiển phương tiện, hai luồng dữ liệu giữa mô phỏng và hiển thị, cùng kết quả thực nghiệm và đánh giá. Nội dung tập trung vào những điểm phản ánh đúng hiện thực của hệ thống thay vì mô tả lý thuyết chung.

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

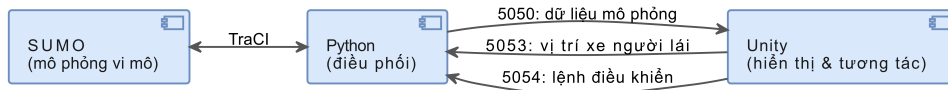
Hệ thống được xây dựng theo kiến trúc phân tầng kết hợp mô hình mô phỏng–hiển thị, trong đó phần mô phỏng giao thông chạy trên SUMO (điều khiển bằng TraCI trong Python) và phần hiển thị, tương tác chạy trên Unity. Hai phần giao tiếp qua kết nối TCP theo cơ chế truyền thông điệp dạng JSON, có dấu hiệu kết thúc gói tin <END> để tách thông điệp trên luồng TCP.

Kiến trúc này được chọn vì các lý do: tách biệt trách nhiệm giữa mô phỏng và hiển thị; dễ mở rộng thêm loại đối tượng hoặc chỉ số thống kê mà ít ảnh hưởng phần còn lại; và tương thích với công cụ, do SUMO đã có TraCI hỗ trợ điều khiển theo bước, còn Unity thuận lợi cho hiển thị thời gian thực và xử lý vật lý. Ở mức khái niệm, có thể ánh xạ theo mô hình Model–View–Controller: phần Model là trạng thái mô phỏng trong SUMO cùng các tệp mạng và tuyến; phần View là các đối tượng và mô-đun dựng cảnh trong Unity; phần Controller là lớp giao tiếp TCP và bộ xử lý lệnh điều khiển.

0.1.2 Kiến trúc tổng quan

Hệ thống gồm ba tầng: tầng mô phỏng (Python kết hợp SUMO/TraCI) khởi tạo mạng, chạy mô phỏng theo bước, trích xuất trạng thái và ghi kết quả; tầng tích hợp (TCP cùng định dạng JSON) đóng gói, tách khung và truyền thông điệp; tầng hiển thị (Unity) nhận dữ liệu, dựng và cập nhật đối tượng, cung cấp giao diện điều khiển và xử lý tương tác lái xe.

Điểm đáng chú ý trong hiện thực là hệ thống dùng ba kênh TCP tách theo mục đích và hai luồng dữ liệu ngược chiều, như minh họa ở Hình 0.1. Kênh chính truyền dữ liệu mô phỏng từ Python sang Unity; kênh cập nhật nhận dữ liệu phương tiện do người dùng lái từ Unity về Python; kênh điều khiển nhận các lệnh tạm dừng, tiếp tục, kết thúc và cập nhật tham số.



Hình 0.1: Kiến trúc chi tiết với ba kênh TCP và hai luồng dữ liệu

0.1.3 Thiết kế các mô-đun chính

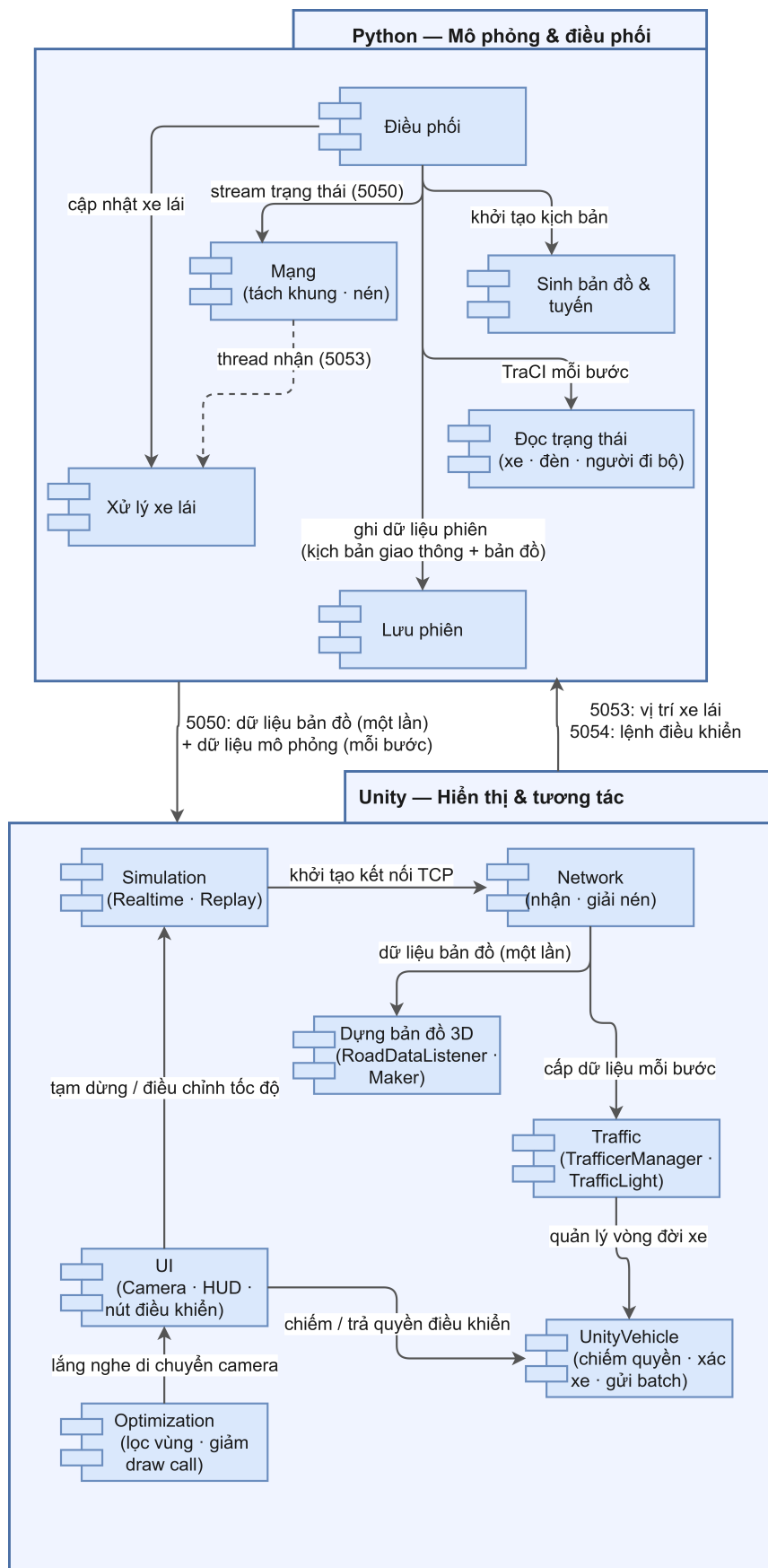
Phía Python đảm nhiệm mô phỏng và điều phối, gồm sáu nhóm mô-đun chính: mô-đun *Điều phối* khởi tạo các kênh TCP, chạy vòng lặp mô phỏng và điều phối toàn bộ luồng xử lý; mô-đun *Mạng* đóng gói, tách khung thông điệp và nén dữ liệu tải xuống; nhóm mô-đun *Đọc trạng thái* trích xuất dữ liệu xe, đèn tín hiệu và người đi bộ từ TraCI sang định dạng JSON chuẩn; mô-đun *Xử lý xe lái* nhận lô dữ liệu từ Unity và phản ánh vào SUMO (mirror/freeze/hủy); nhóm mô-đun *Sinh bản đồ & tuyến* tạo mạng đường và lịch trình di chuyển cho kịch bản mô phỏng; và mô-đun *Lưu phiên* ghi kết quả mỗi lần chạy vào thư mục phiên gồm nhật ký hành trình, bản tóm tắt, dữ liệu mạng đường và chuỗi trạng thái theo bước.

Phía Unity đảm nhiệm hiển thị và tương tác, gồm bảy nhóm mô-đun chính trình bày trong Bảng 1. Trong đó, nhóm mô-đun *Dựng bản đồ 3D* nhận dữ liệu mạng đường từ Python một lần khi khởi chạy và dựng lưới đa giác cho mặt đường, nút giao, vạch sang đường và công trình, tạo nên cảnh ba chiều nền cho toàn bộ phiên mô phỏng.

Bảng 1: Các nhóm mô-đun chính phía Unity

Nhóm mô-đun	Nhiệm vụ
Mạng (Network)	Kết nối TCP, nhận dữ liệu mạng đường và dữ liệu mô phỏng.
Dựng bản đồ 3D (Road)	Nhận dữ liệu mạng đường một lần khi khởi chạy; dựng Mesh mặt đường, nút giao, vạch sang đường và công trình.
Giao thông (Traffic)	Quản lý vòng đời và cập nhật phương tiện, đèn tín hiệu, người đi bộ; nội suy chuyển động.
Phương tiện người lái (UnityVehicle)	Chiếm quyền, lái bằng vật lý, gửi dữ liệu xe lên server, xử lý va chạm.
Tối ưu (Optimization)	Lọc đối tượng theo vùng quan sát, giảm số lệnh vẽ.
Giao diện (UI)	Điều khiển camera, tạm dừng, tốc độ, nút chiếm/trả quyền.
Mô phỏng (Simulation)	Điều phối chế độ thời gian thực và chế độ phát lại.

Cách phân chia mô-đun và quan hệ phụ thuộc giữa hai phía được minh họa ở Hình 0.2.



Hình 0.2: Sơ đồ gói các nhóm mô-đun hai phía và quan hệ phụ thuộc

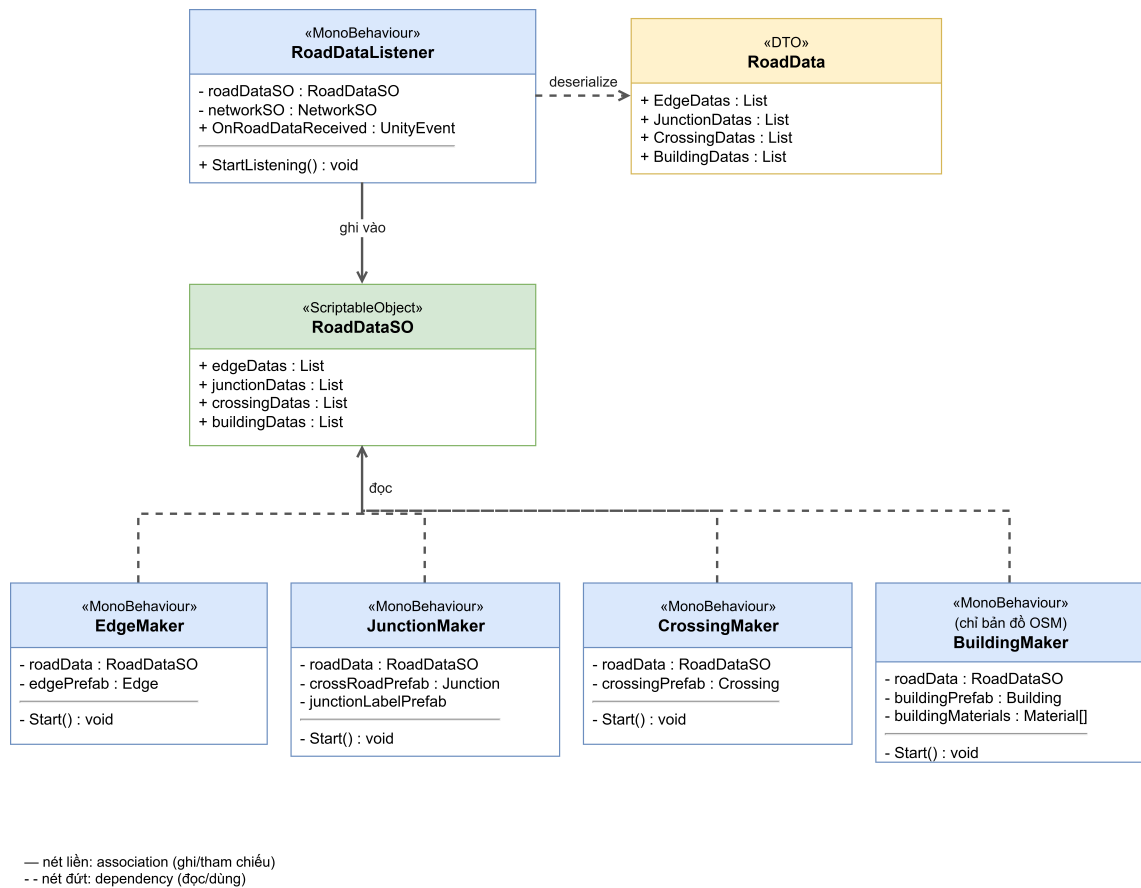
0.2 Thiết kế chi tiết

0.2.1 Thiết kế module dựng bản đồ 3D

Module dựng bản đồ 3D chịu trách nhiệm xây dựng cảnh ba chiều nền — gồm mặt đường, nút giao, vạch sang đường và công trình — từ dữ liệu mạng đường của SUMO. Module này chỉ chạy *một lần* khi khởi chạy, trước khi vòng lặp mô phỏng bắt đầu, và hoạt động hoàn toàn tách biệt với vòng lặp đó.

Phía Python. Khi Unity gửi yêu cầu "RoadDataRequest" qua cổng 5050, `render_map.py` gọi tuần tự bốn nguồn dữ liệu: lớp `EdgeReader` (`edgeType0.py`) trích xuất đa tuyến trung tâm và chiều rộng từng làn đường từ tệp `.net.xml` của SUMO; lớp `CrossRoadReader` (`crossRoad.py`) trích xuất hình dạng đa giác nút giao; lớp `CrossingReader` (`crossing.py`) trích xuất vạch sang đường; và một hàm riêng đọc đa giác công trình từ tệp `OpenStreetMap` nếu bản đồ có nguồn gốc OSM. Toàn bộ dữ liệu được đóng gói thành một gói JSON và gửi về Unity qua cổng 5050; đồng thời ghi ra tệp `road_data.json` trong thư mục phiên để phục vụ kiểm tra.

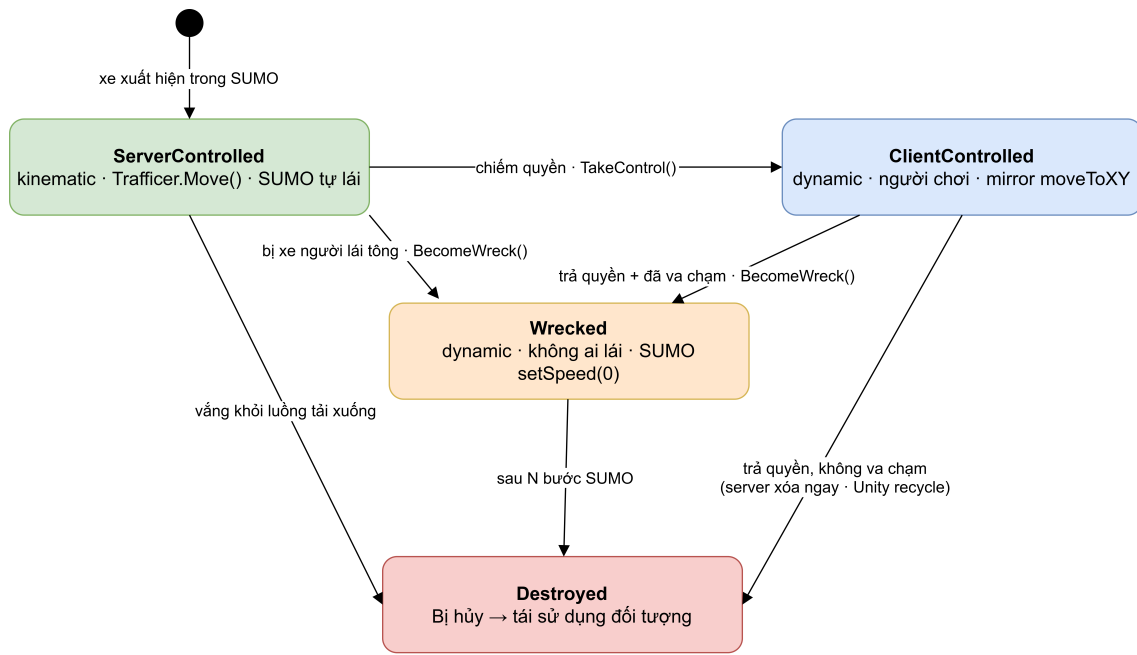
Phía Unity. Hình 0.3 trình bày cấu trúc lớp module phía Unity. `RoadDataListener` nhận gói JSON, giải tuần tự hóa vào lớp trung gian `RoadData`, rồi chép dữ liệu sang `RoadDataSO` — một `ScriptableObject` đóng vai trò kho dữ liệu dùng chung cho bốn lớp `Maker`. Mỗi `Maker` đọc `RoadDataSO` trong `Start()` và dựng Mesh cho một loại đối tượng địa lý: `EdgeMaker` cho mặt đường, `JunctionMaker` cho nút giao, `CrossingMaker` cho vạch sang đường, và `BuildingMaker` cho công trình (chỉ hoạt động với bản đồ OSM). Sau khi bốn `Maker` hoàn tất, cảnh ba chiều nền sẵn sàng để vòng lặp mô phỏng cập nhật phương tiện lên trên. Chi tiết thuật toán dựng Mesh được trình bày trong Chương 5.



Hình 0.3: Sơ đồ lớp module dựng bản đồ 3D phía Unity

0.2.2 Máy trạng thái điều khiển phương tiện

Điểm thiết kế cốt lõi của phần tương tác là một máy trạng thái xác định quyền điều khiển và hành vi của mỗi phương tiện. Mỗi phương tiện luôn ở một trong bốn trạng thái: được SUMO điều khiển, được người dùng điều khiển, trở thành xác xe sau va chạm, hoặc bị hủy. Quyền điều khiển và việc bật/tắt vật lý được quyết định hoàn toàn bởi trạng thái này, thay vì dựa vào sự hiện diện của thành phần. Hình 0.4 mô tả máy trạng thái.



* CLIENT_CAR bắt đầu thẳng ở ClientControlled, không qua ServerControlled

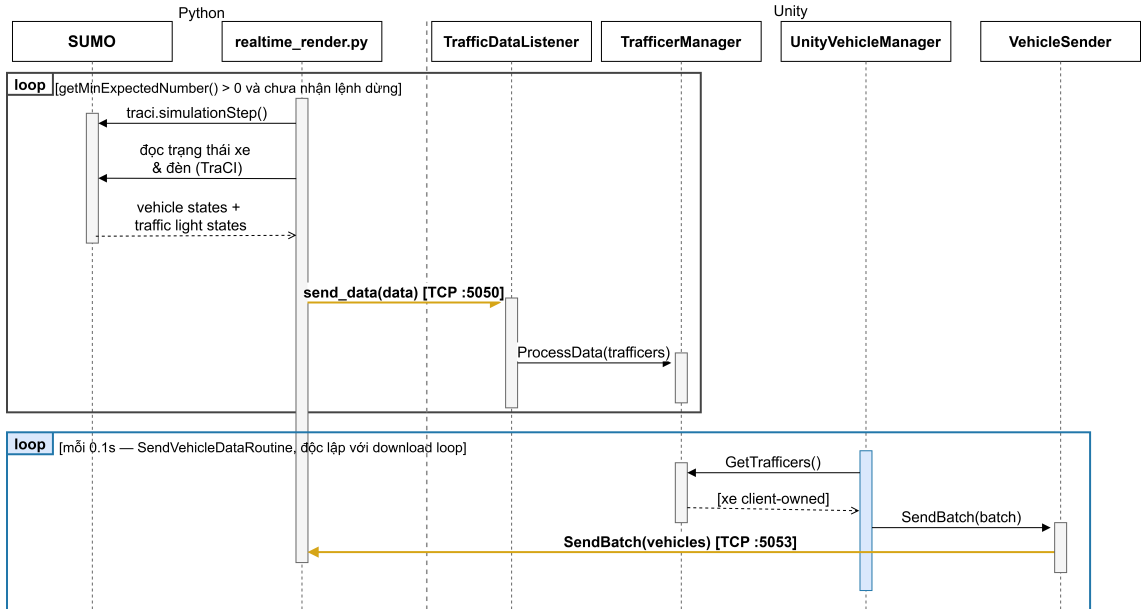
Hình 0.4: Máy trạng thái điều khiển phương tiện

Khi một phương tiện do SUMO điều khiển, vị trí của nó được cập nhật theo dữ liệu mô phỏng và phần vật lý được tắt. Khi người dùng chiếm quyền, hệ thống bật vật lý để người dùng lái; vị trí xe được phản ánh ngược về SUMO để các xe khác phản ứng. Khi một xe đang do người dùng điều khiển hoặc một xác xe va chạm với xe do SUMO điều khiển, xe bị tông chuyển thành xác xe, phía SUMO đặt tốc độ về 0 để tạo vật cản gây ùn ứ trong khi vẫn đồng bộ vị trí từ Unity, và tự bị hủy sau một số bước định trước. Khi trả quyền mà xe không va chạm, server nhận tín hiệu một lần và xóa xe khỏi mô phỏng ngay lập tức; phía Unity tái sử dụng đối tượng khi xe văng khỏi luồng tải xuống. Trạng thái bị hủy cũng đưa phương tiện về bộ nhớ tái sử dụng theo cùng cơ chế.

0.2.3 Hai luồng dữ liệu giữa mô phỏng và hiển thị

Hệ thống có hai luồng dữ liệu ngược chiều chạy hoàn toàn độc lập với nhau. Luồng tải xuống truyền trạng thái mô phỏng từ Python sang Unity sau mỗi bước mô phỏng, gồm chỉ số bước hiện tại, trạng thái đèn tín hiệu và danh sách đối tượng giao thông; trong đó mỗi đối tượng chứa định danh, loại, vị trí, hướng và một số cờ trạng thái. Để giảm băng thông, dữ liệu tải xuống được nén theo thuật toán raw-deflate và mã hóa base64 trước khi gửi; Unity nhận biết gói nén qua tiền tố GZ : và giải nén bằng DeflateStream. Luồng tải lên chạy độc lập theo coroutine mỗi 0,1 giây, không đồng bộ với nhịp tải xuống: UnityVehicleManager lấy danh sách xe do người dùng điều khiển từ TrafficerManager, đóng gói thành một lô duy nhất gồm định danh, vị trí, hướng, tốc độ và trạng thái tồn tại của từng xe,

rồi chuyển cho `VehicleSender` gửi qua cổng 5053. Phía Python dùng luồng tải lên để phản ánh xe người lái vào SUMO và đối soát danh sách phương tiện đang được quản lý. Hình 0.5 minh họa hai luồng này qua hai khung vòng lặp riêng biệt: khung đen là luồng tải xuống, khung xanh là luồng tải lên.



Hình 0.5: Trình tự trao đổi thông điệp giữa mô phỏng và hiển thị trong một bước

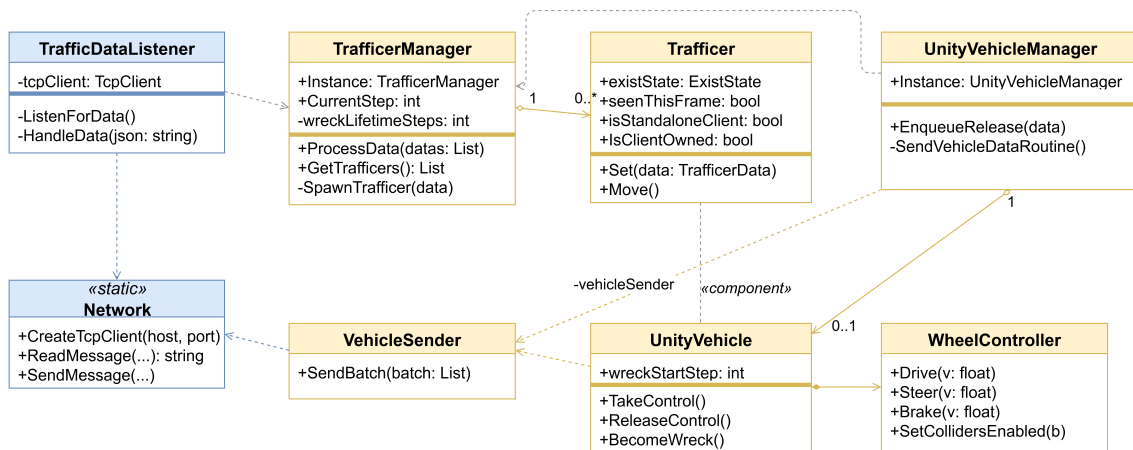
0.2.4 Thiết kế module giao thông

Trong phạm vi báo cáo, một số lớp chủ đạo của module giao thông được trình bày như sau.

- `TrafficManager` (Unity): quản lý vòng đời phương tiện theo dữ liệu tải xuống, tạo, cập nhật và thu hồi đối tượng; đếm tập trung số bước để hủy xác xe nhằm tránh lỗi khi đối tượng bị tạm ẩn.
- `UnityVehicle` (Unity): xử lý chiếm quyền, trả quyền và chuyển xe thành xác xe; bật/tắt vật lý theo trạng thái; thu thập dữ liệu xe để gửi lên.
- `WheelController` (Unity): điều khiển bốn bánh xe gồm tăng tốc, đánh lái và phanh khi xe ở chế độ lái.
- `VehicleSender` (Unity): gửi dữ liệu phương tiện theo lô lên cổng 5053; giữ lô mới nhất để truyền, bỏ qua các lô cũ hơn nếu kịp nhận thêm.
- `UnityVehicleManager` (Unity): điều phối luồng tải lên; coroutine của lớp này cứ 0,1 giây lấy danh sách xe `IsClientOwned` từ `TrafficManager`, đóng gói thành lô và chuyển cho `VehicleSender`; ngoài ra gửi tín hiệu one-shot trả quyền (`state = 1`) để server xóa xe ngay.

- **SimulationSession (Python):** gom toàn bộ kết quả của một lần chạy vào thư mục phiên `result/{bản_đồ}-{thời_điểm}/`; mở tệp `scenario.json` ở chế độ streaming và ghi từng khung trạng thái qua `record_frame()`, tự đẩy đệm khi đạt ngưỡng 10 MB.
- **TrafficerData, TrafficLightData (Python):** lớp dữ liệu đóng gói trạng thái phương tiện và đèn tín hiệu sau mỗi bước mô phỏng; phương thức `to_dict()` chuyển sang JSON để gửi qua TCP.

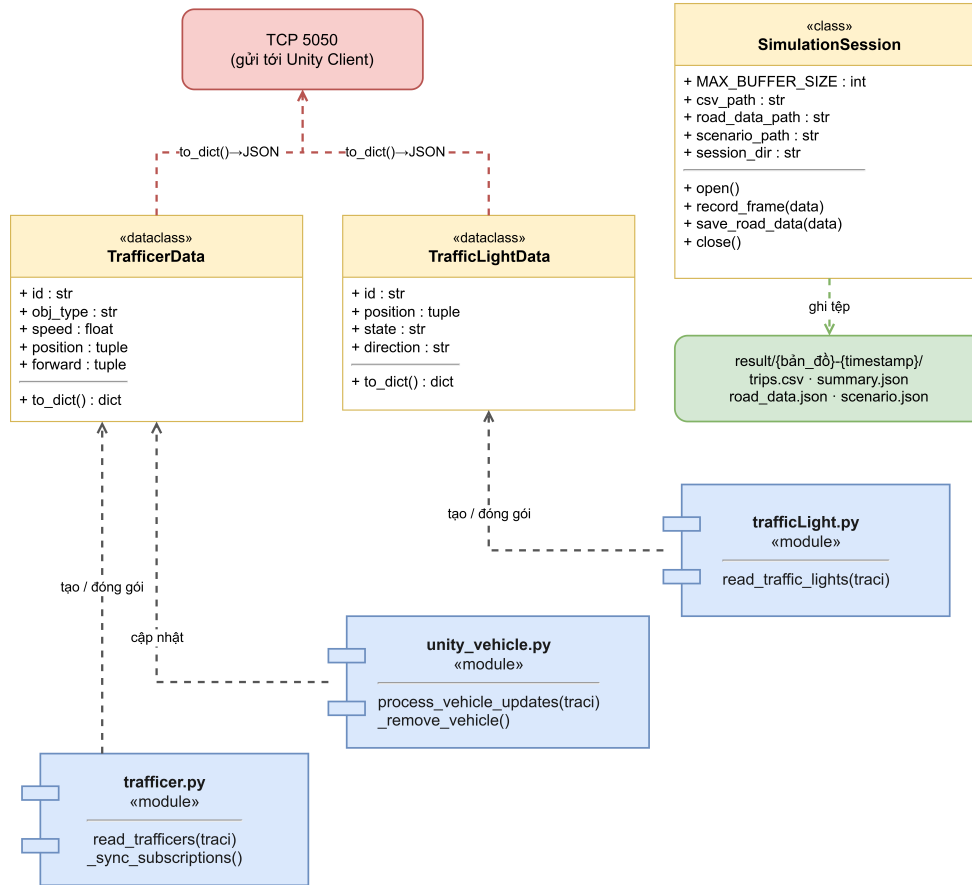
Quan hệ giữa các lớp chủ đạo phía Unity được minh họa ở Hình 0.6.



Hình 0.6: Sơ đồ lớp các lớp chủ đạo phía Unity (nhóm quản lý phương tiện và điều khiển lái).

P phía Python server, các lớp và mô-đun giao thông được tổ chức thành ba nhóm: tầng dữ liệu gồm `TrafficerData` và `TrafficLightData` đóng gói trạng thái từng đối tượng; tầng đọc trạng thái gồm các mô-đun `trafficer.py`, `unity_vehicle.py` và `trafficLight.py` gọi `TraCI` để trích xuất dữ liệu mỗi bước; và lớp `SimulationSession` quản lý toàn bộ đầu ra của một phiên chạy vào thư mục riêng. Hình 0.7 minh họa quan hệ giữa các thành phần này.

Module giao thông phía Python Server



Hộp vàng: class / dataclass Python
 Hộp xanh: module Python (function-level)
 Hộp xanh lá: tệp/thư mục đầu ra
 -- nét đứt: dependency (tạo / dùng / gửi / ghi)

Hình 0.7: Sơ đồ lớp và mô-đun giao thông phía Python server

0.2.5 Lưu trữ dữ liệu

Hệ thống không dùng cơ sở dữ liệu truyền thống mà lưu dữ liệu dưới dạng tệp. Dữ liệu cấu hình và mạng mô phỏng là các tệp của SUMO. Dữ liệu kết quả của mỗi lần chạy được gom vào một thư mục phiên đặt theo tên bản đồ và thời điểm chạy, gồm nhật ký hành trình của xe, bản tóm tắt, dữ liệu mạng đường và chuỗi trạng thái theo từng bước. Cách lưu này phù hợp với mục tiêu mô phỏng, hiển thị và thu thập thống kê theo từng lần chạy, đồng thời cho phép phát lại. Cấu trúc một thư mục phiên tiêu biểu được minh họa ở Hình 0.8.

```

result/
└─ HoanKiem-2026-06-13-09-30-05/
    └─ trips.csv                — nhật ký hành trình của xe
    └─ summary.json            — bản tóm tắt lần chạy
    └─ road_data.json          — dữ liệu mạng đường (nút, cạnh, vạch qua đường)
    └─ scenario.json           — chuỗi trạng thái mô phỏng theo từng bước

```

* trips.csv đổi thành trips-ped.csv khi kịch bản có người đi bộ

Hình 0.8: Cấu trúc một thư mục phiên tiêu biểu, đặt tên theo bản đồ và thời điểm chạy. Tập `trips.csv` đổi thành `trips-ped.csv` khi kịch bản có người đi bộ.

0.2.6 Thiết kế giao diện người dùng

Khác với các ứng dụng hướng biểu mẫu, giao diện của hệ thống được thiết kế xoay quanh việc *quan sát và tương tác với cảnh ba chiều*, nên thành phần trung tâm không phải là các ô nhập liệu mà là khung nhìn (camera) cùng một lớp điều khiển gọn nhẹ phủ lên trên. Giao diện được tổ chức theo nguyên tắc: ưu tiên không gian hiển thị cho cảnh, các bảng điều khiển chỉ hiện khi cần và có thể ẩn đi; thao tác chính dùng chuột và bàn phím; và các nút mang tính ngữ cảnh, chỉ xuất hiện khi thực sự áp dụng được cho đối tượng đang chọn.

Chế độ hiển thị. Hệ thống có hai chế độ với bố cục giao diện riêng: chế độ thời gian thực (hiển thị mô phỏng đang chạy, kèm các lệnh điều khiển quá trình chạy) và chế độ phát lại (đọc lại một phiên đã lưu). Trước khi vào cảnh, người dùng chọn chế độ, bản đồ và tham số mô phỏng ở bước cấu hình khởi chạy.

Camera và góc nhìn. Camera hỗ trợ hai góc nhìn nhằm phục vụ cả nhu cầu giám sát tổng thể lẫn trải nghiệm của người tham gia giao thông:

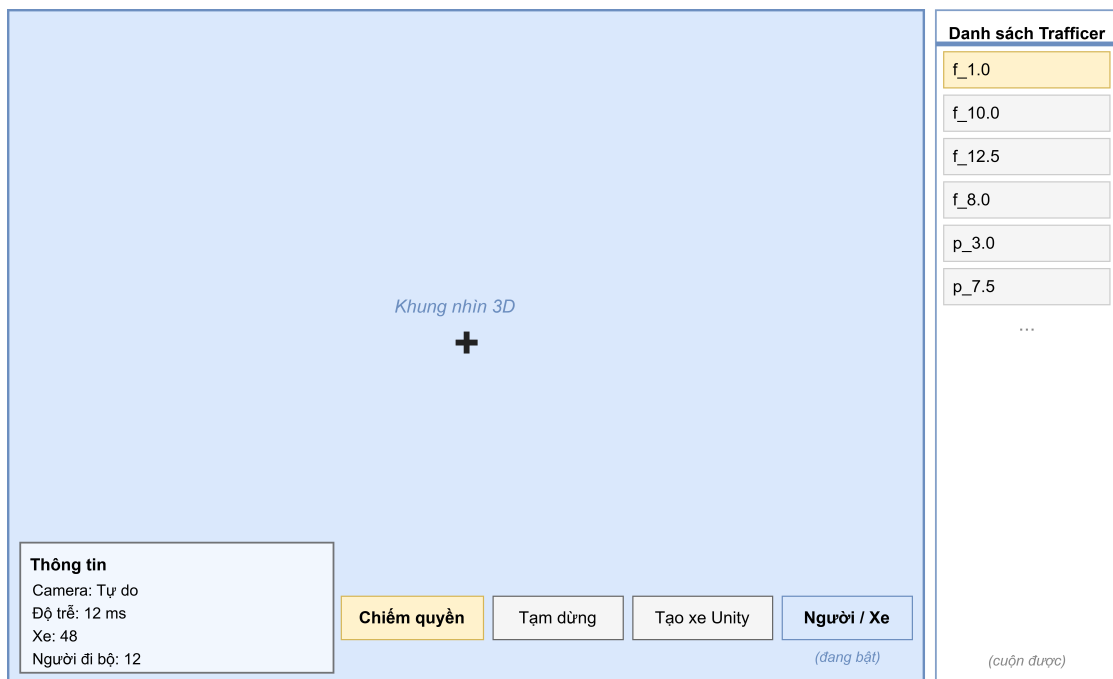
- **Góc nhìn tự do:** camera bay tự do trên cao, di chuyển bằng phím (WASD) và xoay bằng chuột, với tốc độ di chuyển và độ nhạy chuột điều chỉnh được. Đây là góc nhìn để quan sát toàn cảnh mạng đường và dòng giao thông.
- **Góc nhìn gắn vào phương tiện:** camera gắn vào một phương tiện cụ thể để nhìn theo nó. Người dùng chọn phương tiện bằng cách trỏ và bấm (point-and-select), hoặc chọn ngẫu nhiên một phương tiện đang được hiển thị trong danh sách. Đây là góc nhìn tạo nên trải nghiệm theo người tham gia giao thông và là tiền đề cho thao tác chiếm quyền điều khiển.

Trạng thái camera hiện hành (tự do hay đang gắn vào phương tiện nào) được hiển thị thường trực để người dùng định hướng.

Lớp điều khiển và bảng thông tin (HUD). Phủ trên khung nhìn là một lớp giao diện gọn nhẹ. Góc dưới bên trái là bảng thông tin hiển thị trạng thái camera (tự do

hay đang gắn vào phương tiện nào) cùng các chỉ số tổng quan của phiên chạy: độ trễ đầu–cuối, số phương tiện và số người đi bộ. Độ trễ đầu–cuối được đo bằng cách server Python đánh nhãn thời gian vào mỗi gói dữ liệu ngay trước khi gửi; phía Unity tính hiệu số giữa thời điểm nhận và nhãn đó để ra độ trễ của bước vừa hoàn tất. Vì cả hai tiến trình chạy trên cùng một máy nên đồng hồ hệ thống là thống nhất và số đo phản ánh chính xác chi phí xử lý và truyền dữ liệu qua TCP nội bộ. Góc dưới bên phải là hàng nút điều khiển chính gồm: nút *chiếm/trả quyền điều khiển* mang tính ngữ cảnh (chỉ hiện khi camera đang gắn vào một phương tiện có thể lái, và tự đổi nhãn giữa “Chiếm quyền” và “Trả quyền” theo trạng thái xe); nút *tạm dừng/tiếp tục* mô phỏng; nút *Tạo xe Unity* để sinh một phương tiện riêng do người dùng điều khiển (và đổi thành “Hủy xe Unity” để loại bỏ nó); và nút *Người/Xe* để ẩn/hiện danh sách các đối tượng giao thông (phương tiện và người đi bộ) đang có trong cảnh theo mã định danh, chọn một mục trong danh sách sẽ gắn camera vào đúng đối tượng tương ứng. Hình 0.9 minh họa bố cục giao diện trong cảnh.

Ngoài hàng nút chính, các thiết lập ít dùng hơn được đặt trong các bảng tùy chọn mở bằng phím `ESC`, gồm: điều chỉnh tốc độ mô phỏng, tốc độ và độ nhạy của camera, tùy chọn lọc theo vùng quan sát phục vụ tối ưu hiển thị, và lệnh kết thúc phiên chạy. Khi một bảng tùy chọn mở, con trỏ chuột được mở khóa để thao tác và được khóa lại khi đóng để tiếp tục điều khiển camera; ngoài ra bảng hướng dẫn phím điều khiển cũng có thể bật/tắt khi cần.



Hình 0.9: Bố cục giao diện người dùng trong cảnh ba chiều

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Bảng 2 liệt kê các công cụ chính được sử dụng trong dự án.

Bảng 2: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ	Phiên bản
Mô phỏng giao thông vi mô	Eclipse SUMO	1.24.0
Điều khiển mô phỏng	TraCI (Python)	đi kèm SUMO
Ngôn ngữ điều phối	Python	3.x
Hiển thị 3D và tương tác	Unity Editor	6000.3.9f1
Phân tích JSON trong Unity	Newtonsoft.Json	3.2.x
Môi trường phát triển	Visual Studio Code	—
Quản lý phiên bản	Git	—
Hệ điều hành thử nghiệm	Windows	10/11

0.3.2 Kết quả đạt được và thống kê mã nguồn

Sản phẩm chính gồm: bộ sinh mạng và tuyến cho SUMO (từ lưới mê cung hoặc từ bản đồ OpenStreetMap); ứng dụng Unity hiển thị mô phỏng ba chiều có tương tác lái xe; và bộ lưu trữ phiên cùng nhật ký kết quả. Bảng 3 thống kê nhanh quy mô mã nguồn do nhóm tự viết, không tính thư viện bên thứ ba.

Bảng 3: Thống kê quy mô mã nguồn

Thành phần	Số tệp	Số dòng
Python (mô phỏng và điều phối)	40	5775
Unity (mã nguồn C#)	100	8060

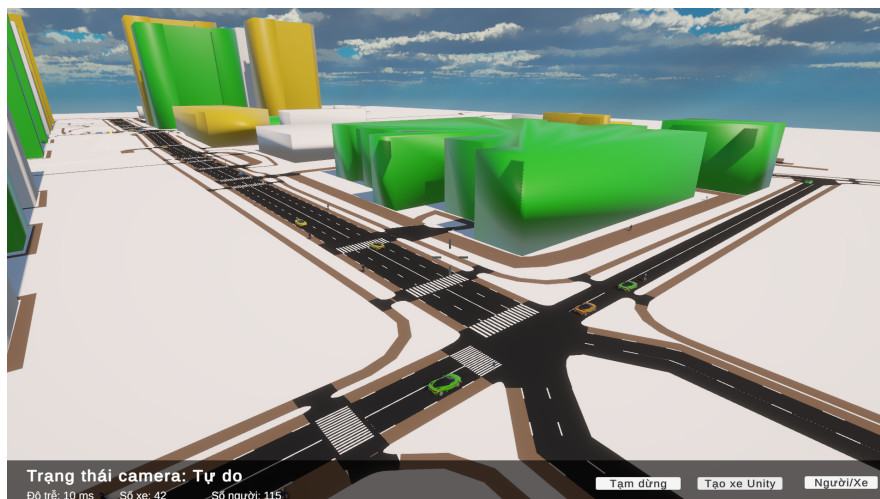
0.3.3 Minh họa các chức năng chính

Các chức năng chính đã hiện thực gồm: sinh mạng đường từ lưới mê cung hoặc bản đồ thực tế; chạy mô phỏng và đồng bộ trạng thái sang Unity theo thời gian thực; hiển thị phương tiện, đèn tín hiệu và người đi bộ trong không gian ba chiều; chiếm quyền điều khiển và lái xe thủ công; xử lý va chạm sinh xác xe và trả quyền xóa xe khỏi mô phỏng; cùng các thao tác điều khiển quá trình chạy. Các hình minh họa dưới đây thể hiện một số chức năng tiêu biểu.

0.4 Kiểm thử

Do hệ thống có tính tích hợp cao giữa SUMO và Unity, việc kiểm thử được thực hiện theo hai hướng: kiểm thử chức năng thủ công theo use case và kiểm thử hộp đen cho các mô-đun xử lý dữ liệu và giao thức. Bảng 4 liệt kê một số trường hợp kiểm thử tiêu biểu.

Trong điều kiện chạy thử nghiệm, các trường hợp trên đạt yêu cầu. Hiện tượng



Hình 0.10: Minh họa cảnh giao thông ba chiều trong Unity: góc nhìn từ trên cao

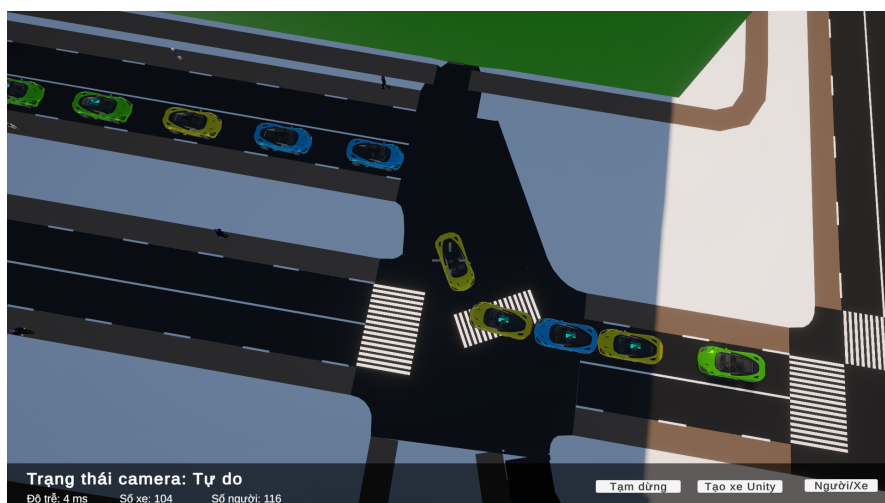


(a) Quan sát xe tự hành, nút “Chiếm quyền” hiện ra



(b) Đang lái xe (camera gắn CLIENT_CAR)

Hình 0.11: Minh họa chế độ lái xe thủ công: từ quan sát đến chiếm quyền điều khiển



Hình 0.12: Minh họa xử lý va chạm: xác xe nằm lại gây ùn ứ dòng giao thông

Bảng 4: Một số trường hợp kiểm thử tiêu biểu

Mã	Mục tiêu	Dữ liệu vào / Kỳ vọng
TC01	Tách khung thông điệp TCP	Gửi hai thông điệp liên tiếp trong một lần gửi; phía nhận tách đúng hai thông điệp.
TC02	Nhận dữ liệu mạng đường một lần	Unity yêu cầu dữ liệu mạng; nhận đủ và dựng cảnh đúng.
TC03	Hiển thị trạng thái đèn	Mô phỏng trả về chuỗi trạng thái đèn; Unity hiển thị đúng đỏ/vàng/xanh.
TC04	Chiếm quyền và lái xe	Người dùng chiếm một xe; xe chuyển sang lái bằng vật lý, các xe khác phản ứng theo.
TC05	Va chạm sinh xác xe	Xe người lái tông xe của mô phỏng; xe bị tông thành xác xe và biến mất sau N bước.
TC06	Trả quyền điều khiển	Thả xe không va chạm: xe bị xóa khỏi mô phỏng ngay, Unity tái sử dụng đối tượng; thả xe đã va chạm: xe chuyển sang xác xe (TC05).

lệch đồng bộ chủ yếu xuất hiện khi tốc độ mô phỏng đặt quá cao; khi điều chỉnh tốc độ hợp lý, quan sát ổn định hơn.

0.5 Thực nghiệm và đánh giá

0.5.1 Môi trường và kịch bản thực nghiệm

Hệ thống được thử nghiệm trên máy cá nhân chạy Windows với hai tiến trình: tiến trình Python kết hợp SUMO và tiến trình Unity. Hai nhóm kịch bản được sử dụng: nhóm kịch bản sinh từ lưới mê cung với kích thước tăng dần nhằm khảo sát khả năng chịu tải theo số lượng đối tượng; và nhóm kịch bản từ bản đồ thực tế OpenStreetMap nhằm kiểm chứng khả năng dựng cảnh và hiển thị trên dữ liệu thật.

0.5.2 Phương pháp đánh giá

Việc đánh giá tập trung vào ba khía cạnh. Thứ nhất là tính đúng đắn của hiển thị, được kiểm chứng bằng cách đối chiếu cảnh ba chiều trong Unity với giao diện hai chiều của SUMO trên cùng một kịch bản. Thứ hai là hiệu năng hiển thị, đo bằng tốc độ khung hình khi số lượng phương tiện thay đổi. Thứ ba là tính ổn định của tương tác, quan sát qua các thao tác chiếm quyền, lái xe, va chạm và trả quyền.

0.5.3 Kết quả

Bảng 5 ghi nhận kết quả hiệu năng trên máy bàn (Intel Core i5-14400F, NVIDIA RTX 4060, 32 GB RAM DDR4) với Unity giới hạn ở 120 FPS. Ba kịch bản mê cung có kích thước lần lượt 32×32 , 64×64 và 128×128 ô, mỗi kịch bản chạy khoảng 100 phương tiện và 100 người đi bộ; kịch bản OSM chạy khoảng 50 phương tiện và 50 người đi bộ.

Bảng 5: Kết quả hiệu năng theo quy mô kịch bản (máy bàn)

Kịch bản	Xe / Người đi bộ	FPS TB	FPS thấp nhất	Độ trễ TB (ms)
Mê cung nhỏ (32×32)	100 / 100	119.5	52.9	6.2
Mê cung vừa (64×64)	100 / 100	112.1	40.6	8.3
Mê cung lớn (128×128)	100 / 100	100.1	11.9	13.0
Bản đồ OSM	50 / 50	119.8	93.3	6.2

Bảng 6 đối chiếu hiệu năng kịch bản OSM trên máy tính xách tay (Dell Inspiron 5620, Intel Core i5-1240P, NVIDIA MX570, 16 GB RAM DDR4).

Bảng 6: Kết quả hiệu năng kịch bản OSM trên máy tính xách tay

Kịch bản	Xe / Người đi bộ	FPS TB	FPS thấp nhất	Độ trễ TB (ms)
Bản đồ OSM	50 / 50	81.5	40.8	11.8

Kết quả cho thấy trên máy bàn, hệ thống duy trì FPS trung bình sát giới hạn 120 với kịch bản mê cung nhỏ (119,5 FPS) và bản đồ OSM (119,8 FPS). Khi quy

mô tăng lên mê cung vừa và lớn, FPS trung bình giảm lần lượt xuống 112,1 và 100,1, trong khi độ trễ truyền thông tăng từ 6,2 ms lên 8,3 và 13,0 ms do SUMO cần nhiều thời gian tính toán hơn mỗi bước mô phỏng trên mạng đường lớn hơn. Bản đồ OSM với 50 phương tiện và 50 người đi bộ đạt FPS thấp nhất là 93,3, ổn định hơn đáng kể so với mê cung 128×128 (FPS thấp nhất 11,9) vốn xuất hiện các đỉnh tải ngấn trong giai đoạn khởi tạo cảnh. Trên máy tính xách tay phổ thông (i5-1240P, MX570), kịch bản OSM đạt 81,5 FPS trung bình, cho thấy hệ thống có thể vận hành ổn định với các kịch bản quy mô thông thường mà không đòi hỏi phần cứng chuyên dụng. Nhìn chung, kiến trúc pipeline đủ đáp ứng yêu cầu hiển thị thời gian thực trên cả hai lớp thiết bị.

0.5.4 Triển khai

Trong phạm vi đồ án, hệ thống chạy theo mô hình một máy và một người dùng. Quy trình chạy gồm các bước: chuẩn bị kịch bản và khởi chạy tiến trình Python kết hợp SUMO thông qua giao diện khởi chạy; sau đó chạy ứng dụng Unity để kết nối, nhận dữ liệu và hiển thị. Ba kênh TCP được dùng tương ứng cho dữ liệu mô phỏng, dữ liệu phương tiện do người dùng lái và lệnh điều khiển. Kết quả mỗi lần chạy được lưu cục bộ theo từng phiên để phục vụ phân tích và phát lại.

Chương này đã trình bày thiết kế kiến trúc ba tầng với ba kênh TCP và hai luồng dữ liệu, máy trạng thái điều khiển phương tiện, thiết kế các mô-đun và lớp chủ đạo, cùng phương pháp kiểm thử, thực nghiệm và triển khai. Các giải pháp kỹ thuật nổi bật phát sinh trong quá trình thiết kế được trình bày chi tiết trong Chương 5.